

Base Integration

Explains basic integration for Jio Payment Solutions Limited Payment Gateway

Environment

To create a test merchant account, connect with the JPSSL Team. After you create a test merchant account, you can access the test Key on Jiopay Dashboard.

Environment	API Endpoint
Test Environment	https://uat.jiopay.co.in/
Production Environment	https://jiopay.co.in/

Security

Jio Payment Solutions Limited implements hashing algorithms to protect the data sharing between the servers during API Call. we have implemented SHA256 algorithm to generate secure hash for the API Calls. SHA256 (Secure Hash Algorithm 256-bit) is a cryptographic hash function that generates a fixed-size 256-bit (32-byte) hash from input data of any size. It is widely used in securing API calls due to its robust security features.

Jio Payment Solutions Limited system expects a secure hash in the request which is validated at our server to ensure data integrity in transit. Our system response also includes the secure hash generated at our server end and merchant should validate the secure hash in response to ensure correctness of data.

By leveraging SHA256, API calls achieve heightened security, reliability, and trustworthiness, making it essential for modern web services.

Refer to the [Generate Hash](#) guide for detailed implementation.

Generate Hash

Guide to generate hash for API calls with Jio Payment Solutions Limited server

A hash is a cryptographically generated checksum included in a payment request you send and returned by Jiopay in the payment response. By utilizing HMAC-SHA256 (Hash-based Message Authentication Code with Secure Hash Algorithm 256-bit), this hash provides robust protection against "man-in-the-middle" attacks. HMAC-SHA256 combines the SHA256 algorithm with a secret key to produce a unique 256-bit hash, ensuring both the integrity and authenticity of transaction data. This makes it highly secure, resistant to tampering, and effective in safeguarding transactions from unauthorized modifications.

Hash generation logic

Jio Payment Solutions Limited uses the SHA-256 hash function that belongs to the SHA-2 family of cryptographic functions to generate hash values.

STEP 1: Getting Transaction Data

To collect the required details for generating hash

- Request Parameters (key-value) pairs
- SECRET_KEY for the merchant, can be generated and downloaded from the JioPay Merchant Dashboard

STEP 2: Creating a message string

To create a string from the request parameter values sort the keys of the request and concatenate the values of all the parameters without any delimiters.

For eg., sample request and generating message string for it

Request - Unsorted

```
{  
  "amount": "100",  
  "requestType": "UPIQR",  
  "currency": "365",  
  "emailID": "test@jiopay.in",  
  "merchantId": "JP2001100060811",  
}
```

```
"merchantRefNo": "TEST4828245774",  
"customerID": "9876543210"  
}
```

Sort the request keys in alphabetical order

Request - Sorted

```
{  
  "amount": "100",  
  "currency": "365",  
  "customerID": "9876543210",  
  "emailID": "test@jiopay.in",  
  "merchantId": "JP2001100060811",  
  "merchantRefNo": "TEST4828245774",  
  "requestType": "UPIQR"  
}
```

Concatenate the key values of sorted request to form message string

```
messageString =  
100365test@jiopay.inJP2001100060811TEST4828245774UPIQR
```

STEP 3: Generating hash from message string

To create the hashed string from the message string created in STEP 2, use the HMAC function with SHA256 encryption algorithm to generate a hash of the concatenated message string.

```
hmac_sha256(100365test@jiopay.inJP2001100060811TEST48282457  
74UPIQR)
```

Convert the result to HEX format (lowercase) - Output Encoding

```
For SECRET_KEY = abc generating the hash value would  
be e745683bc94d4aef61d842cf815351fd78c59de521678da6b74b2ad5  
e2d2c685
```

Final request with secure hash

Request - with Hash

```
{  
  "amount": "100",  
  "currency": "365",  
  "customerID": "9876543210",  
  "emailID": "test@jiopay.in",  
  "hash": "e745683bc94d4aef61d842cf815351fd78c59de521678da6b74b2ad5e2d2c685"
```

```
    "merchantId": "JP2001100060811",
    "merchantRefNo": "TEST4828245774",
    "requestType": "UPIQR",
    "secureHash":
    "e745683bc94d4aef61d842cf815351fd78c59de521678da6b74b2ad5e2
    d2c685"
}
```

Sample Code

Sample Code blocks to generate secure hash using HMAC - SHA256 Algorithm

Java
Python
JavaScript

```
public static String hmacDigest(String msg, String
keyString) {
    String digest = null;
    try {
        SecretKeySpec key = new
SecretKeySpec((keyString).getBytes("UTF-8"), "HmacSHA256");
        Mac mac = Mac.getInstance("HmacSHA256");
        mac.init(key);
        byte[] bytes = mac.doFinal(msg.getBytes("ASCII"));
        StringBuffer hash = new StringBuffer();
        for (int i = 0; i < bytes.length; i++) {
            String hex = Integer.toHexString(0xFF &
bytes[i]);
            if (hex.length() == 1) {
                hash.append('0');
            }
            hash.append(hex);
        }
        digest = hash.toString();
    } catch (UnsupportedEncodingException e) {} catch
(InvalidKeyException e) {} catch (NoSuchAlgorithmException
e) {}
    return digest;
}
```

Callbacks

Webhooks/Callbacks are a fundamental part of building a robust and reliable payment integration. They provide a powerful way for our system to communicate with your application in real time, ensuring you always have the most up-to-date information about your transactions.

What are Webhooks/Callbacks?

Webhooks, also known as callbacks, are a mechanism for your server to receive real-time notifications about events from our payment gateway. Think of them as an automated phone call: instead of your server constantly calling us to ask "Is the payment ready yet?", we call your server as soon as a significant event happens, like a successful payment or a refund. This "push" communication model is far more efficient than the traditional "pull" model of polling.

When you configure a webhook, you provide us with a specific URL on your server. When a payment event occurs, we send an HTTP POST request to that URL. This request contains a JSON/URL encoded payload with all the relevant transaction details, including the status, amount, and unique identifiers. Using webhooks is crucial for updating your database and fulfilling orders, as it guarantees you receive the final and definitive status of a transaction.

Types of Callbacks

In payment processing, there are two primary types of callbacks you will encounter:

1. **Browser-to-Browser (B2B) Callbacks** This is a redirection-based callback. After a customer completes a payment on our payment gateway's page, we redirect their browser back to a specific URL on your website. The redirect URL can include transaction details as query parameters. This url is the `returnURL` in the Sale API.
 - **Purpose:** To provide a seamless user experience by returning the customer to your site. You can use this to show a success or failure page to the customer.
 - **Limitations:** The merchant doesn't get a real-time response with this method as it depends on the user's browser.

- Recommendation: It should be used exclusively for front-end user experience.
- 2. Server-to-Server (S2S) Callbacks (Webhooks) This is the recommended and most secure method for receiving transaction updates. An S2S callback is a direct communication between our server and your server. When an event occurs, our server makes an HTTP POST request directly to the webhook URL you have configured. This URL is configured on the PG System during Merchant Onboarding.
- Purpose: To provide a definitive, tamper-proof, and reliable update on the transaction status. The request comes from our server, not the customer's browser, making it highly secure.
- Benefits: You receive the final status of a transaction, which is essential for accurate order fulfillment, inventory management, and financial reconciliation. It works even if the customer's browser is closed.
- Recommendation: Always use S2S callbacks to update your backend system and confirm the status of a payment. This should be your single source of truth for transaction status.

S2S Callback

JPSL will send an S2S response to the merchant's server URL. The merchant's server URL should be capable of handling the following content types:

application/json

JSON

```
{
  "acqName": "PayPhi",
  "aggregatorID": "JP2001xxx60985",
  "TransmissionDateTime": "20250910123428",
  "paymentInstId": "4XXX XXXX XXXX 1111",
  "cardNetwork": "VISA",
  "customerMobileNo": "98xxxxxxxx",
  "customerEmailID": "abc@gmail.com",
  "paymentSubInstType": "DC",
  "paymentMode": "Card",
  "amount": "1.00",
  "responseCode": "0000",
  "respDescription": "Transaction successful",
}
```

```
"merchantId": "JP200xxx00007",
"merchantTxnNo": "UAT1280835036",
"txnID": "7700206148341",
"paymentDateTime": "20250910123438",
"paymentID": "82189795538",
"oth_charge": false,
"secureHash":
"161aeca47530e1766ecb6da723303bad65a09ebadc1126fb32e18d9f50
d0c216"
}
```

Response On Payment Advice URL

JSON

```
{
  "acqName": "PayPhi",
  "TransmissionDateTime": "20241018134237",
  "paymentInstId": "6XXX XXXX XXXX 3677",
  "cardNetwork": "RuPay",
  "customerEmailID": "chaitanya.somani@test.com",
  "paymentSubInstType": "DC",
  "paymentMode": "Card",
  "amount": "100.00",
  "responseCode": "0000",
  "respDescription": "Transaction successful",
  "merchantId": "JP2001100060874",
  "merchantTxnNo": "Test4916110172",
  "txnID": "7700204186825",
  "paymentDateTime": "20241018134249",
  "paymentID": "55529451079",
  "secureHash":
  "cf400919608792963d9d8594517b58f762b67647f1b00901b86c401799
  22dc7f"
}
```

Return B2B URL response

```
{
  secureHash:
  '8634021f07db64ca7e9df8e00a654ad80ce4f46714195fd0e5eb2e941e
  8409fe',
  amount: '11.00',
  addlParam2: '6e12bc3c-cb96-486c-b9b0-b3664f84cf1c',
  customerEmailID: 'guest@jiopay.in',
}
```

```
paymentMode: 'Card',
respDescription: 'Transaction successful',
TransmissionDateTime: '20251103100636',
oth_charge: 'false',
paymentInstId: 'XXXX XXXX XXXX 1111',
customerMobileNo: '9876543210',
responseCode: '0000',
acqName: 'PayPhi',
cardNetwork: 'VISA',
paymentSubInstType: 'DC',
merchantId: 'JP2012xxxx0031',
paymentID: '41377684470',
merchantTxnNo: 'ICTXN_MHIZ6MEP52YVG9VE',
addlParam1:
'ICORD_MHIZ67DU7YUDVH7E#ICPAY_MHIZ6MEP09CM0QFD',
paymentDateTime: '20251103153649',
txnID: '7700215243116'
}
```